

結果は2番目の量子ビットに書き込まれます。

2桁以上の数の足し算は、半加算器を数珠つなぎにして組み合わせた回路で計算します(図2.4(下))。最初の Toffoli ゲートと CNOT ゲートでまず a と b の足し算の結果を3番目の量子ビットに出力する部分は先述の半加算器と同じです。続く Toffoli ゲートで、下の桁から来た繰り上がりと $a+b$ との足し算によってさらなる繰り上がりが発生するときには4番目のビットを反転します。最後に CNOT ゲートで、下からの繰り上がりと $a+b$ との足し算を実行し、最終的な計算結果を3番目の量子ビットに出力します。古典の論理回路と同様の足し算が可能です。かつ量子版では重ね合わせ状態で入力すると答えも重ね合わせ状態で出力されます。

このように、古典の論理回路でできることはすべて量子回路で実現できますが、これだけでは同じ計算に必要なゲート個数はそれほど大きく変わらず、量子コンピュータだと効率化されるようには見えません。また、Toffoli ゲートの物理実装も古典コンピュータの NAND ゲートのように簡単ではありません。等価な算術演算ができるという意味では量子コンピュータは古典コンピュータの上位互換といえますが、そのご利益はそれほど自明ではありません。

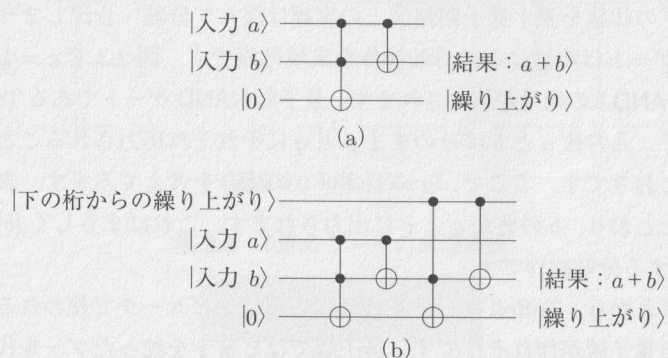


図 2.4 量子版半加算器 (a) と量子版全加算器 (b)

2.3.4 量子コンピュータ版算術論理演算

Toffoli ゲートを使った量子回路によって加算器が構成できたように、古典コンピュータ上のあらゆる計算は量子コンピュータ上でも計算できます。量子コンピュータ版の算術論理演算は Quantum Arithmetic と呼ばれ、関数のような複雑な演算であっても重ね合わせ状態を使って並列計算することが可能です。

例えば、ある量子回路 U_f によって

$$|x\rangle |00\cdots 0\rangle \xrightarrow{U_f} |x\rangle |f(x)\rangle$$

のように、ビット列 x を引数としてビット列を返す関数 $f(x)$ の値を計算して量子ビットとして出力できるとしましょう。関数の計算には補助量子ビットが大量に必要になりますが、ここでは省略しています。 $f(x)$ が古典の論理回路として効率的に実装可能なら、量子回路でも効率的に計算できます。加算器で見たように、算術論理演算について古典と量子の論理回路は等価です。

量子回路による関数 $f(x)$ の計算では、入力には重ね合わせ状態も許されます。例えば、 N 個のビット列の重ね合わせ状態

$$|x\rangle = \frac{1}{\sqrt{N}} \sum_{i=1}^N |x_i\rangle$$

を入力として U_f を実行すると

$$\frac{1}{\sqrt{N}} \sum_{i=1}^N |x_i\rangle |00\cdots 0\rangle \xrightarrow{U_f} \frac{1}{\sqrt{N}} \sum_{i=1}^N |x_i\rangle |f(x_i)\rangle$$

という、ある種の N 並列計算ができます(補助量子ビットは省略)。この状態を測定して演算結果を取り出したいところですが、何も工夫をしなければ N 個ある計算結果は、ボルン規則よりそれぞれ $1/N$ の確率でランダムに測定されるだけで、1回の測定で N 個すべての計算結果を取り出すことはできません。

$f(x^*) = 0$ になる x^* を探す探索問題も、すべての可能な x_i を重ね合わせ状態として入力し量子回路の並列計算による総当たりで簡単に解けそうに思えますが、うまくいきません。なぜなら、解となる結果もそうでない結果も均一に $1/N$ の確率で測定されるので、確率的には解があってもよい N 個の中から解を見つけられません。このように、重ね合わせ状態を使った並列計算が可能というだけでは、量子コンピュータが古典より高速ということにはならないのです。